Résumé TD Algorithmiques

Piles, Files, Listes Chaînées, Récursivité

Préparation Examen – ING3

---

1 Rappels Fondamentaux

Structures de données abstraites

- **PILE (Stack)** : Collection LIFO (Last In, First Out)
  - Utilisée pour : récursivité non terminale, évaluation d'expressions
  - Opérations : `initPile`, `empiler`, `depiler`, `pileVide`, `pilePleine`
- **FILE (Queue)** : Collection FIFO (First In, First Out)
  - Utilisée pour : faire patienter les données, gestion de processus
  - Opérations : `initFile`, `enfiler`, `defiler`, `fileVide`, `filePleine`
- **LISTE CHAÎNÉE** : Collection dynamique avec pointeurs
  - Avantage : taille variable, insertion/suppression O(1) en tête

2 Déclarations de Types – À Mémoriser !

2.1 Pile Statique (Tableau)

Déclaration Pile Statique

```
constante max = 100
type Pile = enregistrement
                T : tableau[1..max] de entier
                N : entier
            fin
var P : PILE
```

2.2 Pile Dynamique (Liste Chaînée)

Déclaration Pile Dynamique

```
type PILE = ^ELEMENT
type ELEMENT = enregistrement
                   val : typeElement
                   suiv : PILE
               fin
var P : PILE
```

2.3 File Statique (Tableau Circulaire)

Déclaration File Statique

```
constante max = 100
type File = enregistrement
                T : tableau[1..max] de entier
                debut, fin : entier
            fin
var F : FILE
```

2.4 File Dynamique (Liste à Deux Entrées)

Déclaration File Dynamique

```
type POINTEUR = ^ELEMENT
type FILE = enregistrement
                tete, queue : POINTEUR
            fin
type ELEMENT = enregistrement
                   val : entier
                   suiv : POINTEUR
               fin
var F : FILE
```

2.5 Liste Chaînée Simple

Déclaration Liste Chaînée

```
type PTR = ^NOEUD
type NOEUD = enregistrement
                 val : entier
                 suiv : PTR
             fin
var tete : PTR
```

3 Implémentation des Primitives

3.1 Primitives Pile Dynamique

InitPile

```
procedure InitPile(r P: PILE)
debut
    P <-- nil
fin
```

Empiler

```
procedure Empiler(d/r P: PILE, d x: typeElement)
var q : PILE
debut
    allouer(q)
    q^.val <-- x
    q^.suiv <-- P
    P <-- q
fin
```

Dépiler

```
procedure Depiler(d/r P: PILE, r x: typeElement)
var q : PILE
debut
    q <-- P
    P <-- P^.suiv
    x <-- q^.val
    liberer(q)
fin
```

PileVide

```
fonction PileVide(d P: PILE) : booleen
debut
    retourner (P = nil)
fin
```

3.2 Primitives File Dynamique

InitFile

```
procedure InitFile(r F: FILE)
debut
    F.tete <-- nil
    F.queue <-- nil
fin
```

Enfiler

```
procedure Enfiler(d/r F: FILE, d x: typeElement)
var q : POINTEUR
debut
    allouer(q)
    q^.val <-- x
    q^.suiv <-- nil

    si fileVide(F)
    alors   F.tete <-- q
            F.queue <-- q
    sinon   F.queue^.suiv <-- q
            F.queue <-- q
    finsi
fin
```

Défiler

```
procedure Defiler(d/r F: FILE, r x: typeElement)
var q : POINTEUR
debut
    q <-- F.tete
    x <-- q^.val
    F.tete <-- q^.suiv
    liberer(q)

    si F.tete = nil
    alors initFile(F)
    finsi
fin
```

FileVide

```
fonction FileVide(d F: FILE) : booleen
debut
    retourner (F.tete = nil)
fin
```

4 Exercices Corrigés – Piles et Files

4.1 Exercice 1 : Reconnaissance de chaîne S*S^{inv}

Énoncé

On se propose de reconnaître les chaînes définies par : une chaîne S suivie du caractère *, suivie de la chaîne S inversée. S ne contient pas '*'.
Exemple : `ab2c*c2ba` est valide.

Correction

```
fonction estValide(d chaine c) : booleen
var P : PILE
    i : entier
    car, x : caractere
debut
    initPile(P)
    i <-- 1

    // Empiler tous les caractères avant '*'
    tantque c[i] <> '*' faire
        empiler(P, c[i])
        i <-- i + 1
    fintantque

    i <-- i + 1  // Sauter le '*'

    // Comparer avec la partie après '*'
    tantque (i <= longueur(c)) et (non pileVide(P)) faire
        depiler(P, x)
        si x <> c[i]
        alors retourner faux
        finsi
        i <-- i + 1
    fintantque

    retourner pileVide(P) et (i > longueur(c))
fin
```

4.2 Exercice 2 : Pairs sous Impairs

Énoncé

On se donne une pile P1 contenant des entiers positifs. Écrire une procédure déplaçant les entiers de P1 vers P2 de façon à avoir dans P2 tous les entiers pairs en dessous des impairs.

Correction – Version Itérative

```
procedure pairsImpairs(d P1 : PILE, r P2 : PILE)
var aux : PILE
    x : entier
debut
    initPile(P2)
    initPile(aux)

    // Séparer pairs (dans P2) et impairs (dans aux)
    tantque non pileVide(P1) faire
        depiler(P1, x)
        si x mod 2 = 0
        alors empiler(P2, x)
        sinon empiler(aux, x)
        finsi
    fintantque

    // Mettre les impairs au-dessus des pairs
    tantque non pileVide(aux) faire
        depiler(aux, x)
        empiler(P2, x)
    fintantque
fin
```

Correction – Version Récursive

```
procedure pairsImpairs(d P1 : PILE, r P2 : PILE)
debut
    initPile(P2)
    TOTO(P1, P2)
fin

procedure TOTO(d/r P1 : PILE, d/r P2 : PILE)
var x : entier
debut
    si non pileVide(P1)
    alors   depiler(P1, x)
            si x mod 2 = 0
            alors   empiler(P2, x)
                    TOTO(P1, P2)
            sinon   TOTO(P1, P2)
                    empiler(P2, x)
            finsi
    finsi
fin
```

4.3 Exercice 3 : Palindrome avec Pile

Énoncé

Un palindrome est un mot qui se lit de la même façon de gauche à droite et de droite à gauche. Exemples : RADAR, ELLE, 1991, ETE.

Correction

```
fonction estPalindrome(d chaine c) : booleen
var P : PILE
    i, n : entier
    x : caractere
debut
    initPile(P)
    n <-- longueur(c)

    // Empiler la première moitié
    pour i de 1 a (n div 2) faire
        empiler(P, c[i])
    finpour

    // Démarrer comparaison (sauter le milieu si impair)
    si n mod 2 = 1
    alors i <-- (n div 2) + 2
    sinon i <-- (n div 2) + 1
    finsi

    // Comparer avec la seconde moitié
    tantque (i <= n) et (non pileVide(P)) faire
        depiler(P, x)
        si x <> c[i]
        alors retourner faux
        finsi
        i <-- i + 1
    fintantque

    retourner vrai
fin
```

4.4 Exercice 4 : Évaluation Expression Postfixée

Énoncé

Écrire un algorithme qui évalue une expression postfixée.
Exemple : `3 7 * 2 22 18 - * +` $\rightarrow$ 29

Fonctions Auxiliaires

```
fonction opBinaire(d c : caractere) : booleen
debut
    retourner c dans ['+', '*', '/', '-']
fin

fonction operer(d c : caractere, d x, y : reel) : reel
debut
    cas c
        '+' : retourner y + x
        '-' : retourner y - x
        '*' : retourner y * x
        '/' : retourner y / x
    fincas
fin
```

Correction Principale

```
fonction evaluerPostfixe(d exp : tab, d L : entier) : reel
var P : pile
    i : entier
    x, y, val : reel
debut
    initPile(P)

    pour i de 1 a L faire
        si opBinaire(exp[i])
        alors   depiler(P, x)
                depiler(P, y)
                val <-- operer(exp[i], x, y)
                empiler(P, val)
        sinon   si opUnaire(exp[i])
                alors   depiler(P, x)
                        val <-- -x
                        empiler(P, val)
                sinon   empiler(P, exp[i])
                finsi
        finsi
    finpour

    depiler(P, val)
    retourner val
fin
```

Ordre de dépilage pour opérateurs

Attention ! Pour une opération $a - b$ en postfix `a b -` :

- On dépile d'abord `b` (dans x)
- Puis on dépile `a` (dans y)
- Résultat = `y - x` = $a - b$

C'est pourquoi `operer` fait `y - x` et non `x - y` !

4.5 Exercice 5 : Inverser une File

Énoncé

Écrire la procédure `inverser(var F : FILE)` qui inverse les éléments de la file F.

Version Récursive (sans structure auxiliaire)

```
procedure inverser(d/r F : FILE)
var x : entier
debut
    si non fileVide(F)
    alors
        defiler(F, x)
        inverser(F)
        enfiler(F, x)
    finsi
fin
```

Version Itérative (avec pile auxiliaire)

```
procedure inverser(d/r F : FILE)
var x : entier
    aux : PILE
debut
    initPile(aux)

    // Vider la file dans la pile
    tantque non fileVide(F) faire
        defiler(F, x)
        empiler(aux, x)
    fintantque

    // Vider la pile dans la file
    tantque non pileVide(aux) faire
        depiler(aux, x)
        enfiler(F, x)
    fintantque
fin
```

4.6 Exercice 6 : File avec Deux Piles

Énoncé

Implémenter une file à l'aide de deux piles P1 et P2.

Déclaration du Type

```
type TFILE = enregistrement
                 P1, P2 : TPILE
             fin
```

Procédures

```
procedure initFile(r F : TFILE)
debut
    initPile(F.P1)
    initPile(F.P2)
fin

procedure enfiler(d/r F : TFILE, d x : typeElement)
debut
    empiler(F.P1, x)
fin

procedure defiler(d/r F : TFILE, r x : typeElement)
var y : typeElement
debut
    // Si P2 est vide, transférer tout de P1 vers P2
    si pileVide(F.P2)
    alors
        tantque non pileVide(F.P1) faire
            depiler(F.P1, y)
            empiler(F.P2, y)
        fintantque
    finsi

    depiler(F.P2, x)
fin

fonction fileVide(d F : TFILE) : booleen
debut
    retourner pileVide(F.P1) et pileVide(F.P2)
fin
```

Trace d'exécution – À savoir refaire !

Opérations : init → enfiler(a,b,c) → défiler → enfiler(d,e) → défiler×2 → défiler

Étape	P1	P2	Retour
1. init	∅	∅	–
2. enfiler(a,b,c)	c,b,a	∅	–
3. défiler	∅	c,b	a
4. enfiler(d,e)	e,d	c,b	–
5. défiler×2	e,d	∅	b, c
6. défiler	∅	e	d

5 Exercices Corrigés – Listes Chaînées

5.1 Création de Liste (Itérative, N connu)

Correction

```
procedure creerListe(r tete : PTR, d n : entier)
var q, dernier : PTR
    i, x : entier
debut
    tete <-- nil

    pour i de 1 a n faire
        allouer(q)
        lire(x)
        q^.val <-- x
        q^.suiv <-- nil

        si tete = nil
        alors   tete <-- q
                dernier <-- q
        sinon   dernier^.suiv <-- q
                dernier <-- q
        finsi
    finpour
fin
```

5.2 Affichage Liste

Affichage Ordre Direct – Itératif

```
procedure afficher(d tete : PTR)
var p : PTR
debut
    p <-- tete
    tantque p <> nil faire
        ecrire(p^.val)
        p <-- p^.suiv
    fintantque
fin
```

Affichage Ordre Direct – Récursif

```
procedure afficherRec(d p : PTR)
debut
    si p <> nil
    alors
        ecrire(p^.val)
        afficherRec(p^.suiv)
    finsi
fin
```

Affichage Ordre Inverse – Récursif

```
procedure afficherInv(d p : PTR)
debut
    si p <> nil
    alors
        afficherInv(p^.suiv)
        ecrire(p^.val)
    finsi
fin
```

5.3 Insertion dans Liste Triée

Procédures pour Insertion Triée

```
procedure creationElt(r q : PTR)
var x : entier
debut
    allouer(q)
    lire(x)
    q^.val <-- x
    q^.suiv <-- nil
fin

procedure rechercherEltAvant(d tete : PTR, d val : entier, r p : PTR)
debut
    p <-- nil
    si tete <> nil
    alors
        si tete^.val < val
        alors
            p <-- tete
            tantque (p^.suiv <> nil) et (p^.suiv^.val < val) faire
                p <-- p^.suiv
            fintantque
        finsi
    finsi
fin

procedure ajoutDebut(d/r tete : PTR, d q : PTR)
debut
    q^.suiv <-- tete
    tete <-- q
fin

procedure ajoutApres(d p : PTR, d q : PTR)
debut
    q^.suiv <-- p^.suiv
    p^.suiv <-- q
fin
```

5.4 Tri Bulles sur Liste Chaînée

Correction

```
procedure triBullesListe(d tete : PTR)
var p : PTR
    echange : booleen
    temp : entier
debut
    repeter
        echange <-- faux
        p <-- tete

        tantque p^.suiv <> nil faire
            si p^.val > p^.suiv^.val
            alors
                // Échanger les valeurs (pas les noeuds)
                temp <-- p^.val
                p^.val <-- p^.suiv^.val
                p^.suiv^.val <-- temp
                echange <-- vrai
            finsi
            p <-- p^.suiv
        fintantque

    jusqua non echange
fin
```

6 Exercices Corrigés – Récursivité

6.1 Fonction d'Ackermann

Correction

```
fonction Ackermann(d m, n : entier) : entier
debut
    si m = 0
    alors retourner n + 1
    sinon si n = 0
          alors retourner Ackermann(m - 1, 1)
          sinon retourner Ackermann(m - 1, Ackermann(m, n - 1))
          finsi
    finsi
fin
```

6.2 Puissance Rapide

Version Simple x^n

```
fonction puissance(d x : reel, d n : entier) : reel
debut
    si n = 0
    alors retourner 1
    sinon retourner x * puissance(x, n - 1)
    finsi
fin
```

Version Optimisée (Exponentiation Rapide)

```
fonction puissanceRapide(d x : reel, d n : entier) : reel
var temp : reel
debut
    si n = 0
    alors retourner 1
    sinon
        temp <-- puissanceRapide(x, n div 2)
        si n mod 2 = 0
        alors retourner temp * temp
        sinon retourner x * temp * temp
        finsi
    finsi
fin
```

6.3 Fibonacci

Correction

```
fonction Fib(d n : entier) : entier
debut
    si n = 0
    alors retourner 0
    sinon si n = 1
          alors retourner 1
          sinon retourner Fib(n - 1) + Fib(n - 2)
          finsi
    finsi
fin
```

6.4 Palindrome Récursif

Correction

```
fonction estPalindromeRec(d c : chaine, d g, d : entier) : booleen
debut
    si g >= d
    alors retourner vrai
    sinon
        si c[g] <> c[d]
        alors retourner faux
        sinon retourner estPalindromeRec(c, g + 1, d - 1)
        finsi
    finsi
fin

// Appel : estPalindromeRec(chaine, 1, longueur(chaine))
```

6.5 Inverser un Tableau Récursivement

Correction

```
procedure inverserTab(d/r T : tableau, d g, d : entier)
var temp : entier
debut
    si g < d
    alors
        temp <-- T[g]
        T[g] <-- T[d]
        T[d] <-- temp
        inverserTab(T, g + 1, d - 1)
    finsi
fin
```

6.6 Somme des Éléments d'un Tableau

Correction

```
fonction sommeTab(d T : tableau, d n : entier) : reel
debut
    si n = 0
    alors retourner 0
    sinon retourner T[n] + sommeTab(T, n - 1)
    finsi
fin
```

6.7 Nombre d'Occurrences

Correction

```
fonction nbOccurrences(d T : tableau, d n, x : entier) : entier
debut
    si n = 0
    alors retourner 0
    sinon
        si T[n] = x
        alors retourner 1 + nbOccurrences(T, n - 1, x)
        sinon retourner nbOccurrences(T, n - 1, x)
        finsi
    finsi
fin
```

6.8 Suite Croissante

Correction

```
fonction estCroissante(d A : tableau, d i, M : entier) : booleen
debut
    si i >= M
    alors retourner vrai
    sinon
        si A[i] > A[i + 1]
        alors retourner faux
        sinon retourner estCroissante(A, i + 1, M)
        finsi
    finsi
fin

// Appel : estCroissante(A, 1, M)
```

7 Exercices Supplémentaires – Séance 17/12

7.1 Tableaux Identiques

Énoncé

Écrire une fonction qui vérifie si deux tableaux T1[N] et T2[M] sont identiques.

Correction

```
fonction tableauxIdentiques(d T1 : tableau, d N : entier,
                            d T2 : tableau, d M : entier) : booleen
var i : entier
debut
    si N <> M
    alors retourner faux
    finsi

    pour i de 1 a N faire
        si T1[i] <> T2[i]
        alors retourner faux
        finsi
    finpour

    retourner vrai
fin
```

7.2 Suppression Colonnes Redondantes

Énoncé

Écrire une procédure qui supprime toutes les colonnes redondantes d'une matrice A[N,M].

Correction

```
fonction colonnesIdentiques(d A : matrice, d N, j1, j2 : entier) : booleen
var i : entier
debut
    pour i de 1 a N faire
        si A[i, j1] <> A[i, j2]
        alors retourner faux
        finsi
    finpour
    retourner vrai
fin

procedure supprimerColonne(d/r A : matrice, d N : entier, d/r M : entier,
                           d col : entier)
var i, j : entier
debut
    pour j de col a M - 1 faire
        pour i de 1 a N faire
            A[i, j] <-- A[i, j + 1]
        finpour
    finpour
    M <-- M - 1
fin

procedure supprimerColonnesRedondantes(d/r A : matrice, d N : entier,
                                       d/r M : entier)
var j, k : entier
debut
    j <-- 1
    tantque j < M faire
        k <-- j + 1
        tantque k <= M faire
            si colonnesIdentiques(A, N, j, k)
            alors supprimerColonne(A, N, M, k)
            sinon k <-- k + 1
            finsi
        fintantque
        j <-- j + 1
    fintantque
fin
```

8 Astuces d'Examen – À Retenir !

Erreurs Fréquentes à Éviter

1. **Ne jamais oublier** `initPile(P)` **ou** `initFile(F)` avant utilisation
2. **Toujours tester** `pileVide(P)` **ou** `fileVide(F)` avant `depiler`/`defiler`
3. **Ordre de dépilage :** pour $a - b$ en postfix, on dépile b puis a, résultat $= a - b$
4. **Pointeurs :** ne jamais oublier `allouer(q)` avant d'utiliser `q^.val`
5. **Récursivité :** toujours définir le **cas de base** en premier

Signatures des Procédures – Notation d/r

- `d` (donnée) : paramètre en entrée seulement
- `r` (résultat) : paramètre en sortie seulement
- `d/r` : paramètre en entrée ET sortie (modifié)
- `var` : équivalent à `d/r`

Exemples :

- `procedure empiler(d/r P: PILE, d x: typeElement)`
  → P est modifié, x est juste lu
- `procedure depiler(d/r P: PILE, r x: typeElement)`
  → P est modifié, x reçoit la valeur dépilée

Conversions Infix → Postfix

Priorités : `^` > `*`, `/` > `+`, `-`
Algorithme :

1. Opérande → sortie directe
2. `(` → empiler
3. `)` → dépiler jusqu'à `(`
4. Opérateur → dépiler tant que priorité sommet ≥ priorité courante, puis empiler
5. Fin → vider la pile

Exemples à connaître :

- $2 + 3 \rightarrow$ `2 3 +`
- $2 * (3 - 5) \rightarrow$ `2 3 5 - *`
- $(2 + 3) * (4 - 1) \rightarrow$ `2 3 + 4 1 - *`

Récursivité : Schéma Type

```
fonction/procedure nomRec(parametres)
debut
    si (CAS DE BASE)
    alors (traitement simple / retourner valeur)
    sinon (traitement + appel récursif sur sous-problème)
    finsi
fin
```

Questions à se poser :

1. Quel est le cas de base ? (arrêt de la récursion)
2. Comment réduire le problème ? (appel sur n-1, sur sous-chaîne, etc.)
3. Où placer le traitement ? (avant ou après l'appel récursif)

Complexité Rapide

Opération	Complexité
Empiler/Dépiler (pile dynamique)	$O(1)$
Enfiler/Défiler (file dynamique)	$O(1)$
Insertion en tête (liste chaînée)	$O(1)$
Recherche dans liste	$O(n)$
Tri bulles	$O(n^2)$
Calcul des rangs (skieurs)	$O(n^2)$
Inversion file (avec pile)	$O(n)$
Fibonacci récursif	$O(2^n)$ – inefficace !
Puissance rapide	$O(\log n)$

Check-list Avant de Rendre

- ☐ Déclarations des types complètes ?
- ☐ Variables locales déclarées ?
- ☐ Initialisation des structures (pile, file, liste) ?
- ☐ Cas de base pour les fonctions récursives ?
- ☐ `debut` et `fin` bien présents ?
- ☐ Libération mémoire (`liberer`) après dépilage ?
- ☐ Retour de valeur pour les fonctions ?

Bon courage pour l'examen !